Software Design Document (SDD)

Module	GPIO Driver
Version	1.0.0
Team	Carlos Araiza · Carlos Cayetano · Kheara Kieley · Vania Leal
Date	26/04/2026
Institution	CETYS Universidad

---

1. Introduction

1.1. Purpose

This document describes the design of the GPIO driver module for the STM32F4xx microcontroller family, based on the requirements in SRS-GPIO_Driver. It covers the register-level logic behind each public function, how input validation works internally, and the AFR configuration sequence added in version 1.1 to support alternate function output.

Read this document to understand how gpio_driver.c works, trace a call from led.c or pwm.c down to the hardware registers, or extend the driver with additional features such as interrupt support or open-drain mode.

1.2. Scope

The GPIO driver abstracts the STM32F4xx GPIO peripheral registers and gives higher-level modules a typed API to enable port clocks, set pin modes, drive outputs, toggle pins, read inputs, and configure alternate functions.

In the lab application: led.c uses gpio_initPort(), gpio_setPinMode(), gpio_clearPin(), gpio_setPin(), and gpio_togglePin() to control the user LED on PA5. button.c uses gpio_initPort(), gpio_setPinMode(), and gpio_readPin() to read the user button on PC13. pwm.c uses gpio_initPort() and gpio_setAlternateFunction() to route TIM3 CH1 to PA6 (AF2) before the PWM signal is enabled. main.c calls gpio_init() once at startup.

1.3. Definitions and Acronyms

Term	Definition
GPIO	General Purpose Input/Output. A configurable digital I/O pin on the MCU that can be set as input, output, alternate function, or analog.
MCU	Microcontroller Unit. Here: STM32F411RE on the Nucleo-F411RE board.
CMSIS	Cortex Microcontroller Software Interface Standard. ARM's vendor-neutral hardware abstraction. Provides GPIO_TypeDef, RCC_TypeDef, and macros like GPIOA, RCC_AHB1ENR_GPIOAEN, and AFR via stm32f4xx.h.
API	Application Programming Interface. The public functions and types declared in gpio_driver.h.
SRS	Software Requirements Specification. Defines what the driver must do.
SDD	Software Design Document. This document. Defines how the driver does it.
RCC	Reset and Clock Control. Manages peripheral clocks. A port whose clock is off ignores all register writes.
AHB1ENR	AHB1 Peripheral Clock Enable Register in RCC. One bit per port: bit 0 = GPIOA, bit 1 = GPIOB, bit 2 = GPIOC, bit 3 = GPIOD, bit 4 = GPIOE.
MODER	Mode Register. Two bits per pin at positions [2*pin+1 : 2*pin]. Values: 00 = input, 01 = output, 10 = alternate function, 11 = analog (reset default). Offset 0x00.
BSRR	Bit Set/Reset Register. Write-only. Bits 0-15 drive a pin HIGH; bits 16-31 drive it LOW. A single 32-bit write is atomic, no read-modify-write needed. Offset 0x18.

ODR	Output Data Register. Bits 0-15 hold the current output state. Toggling a pin via ODR is a read-modify-write and is not atomic. Offset 0x14.
IDR	Input Data Register. Read-only. Reflects the actual pin voltage at the instant of the read. Only bits 0-15 are valid. Offset 0x10.
AFR	Alternate Function Register. Two 32-bit registers per port: AFR[0] covers pins 0-7 (4 bits each), AFR[1] covers pins 8-15. Written to select which peripheral (TIM, USART, SPI, etc.) uses the pin.
AF	Alternate Function. A numbered mapping (0-15) that connects a GPIO pin to a specific peripheral signal. AF2 on PA6 maps to TIM3 CH1.
HAL	Hardware Abstraction Layer. Generic term for a software layer that hides register-level details from upper-layer code.
Active LOW	A signal that is asserted when the line is at 0 V. The Nucleo user button on PC13 reads LOW when pressed.
Push-pull	Output drive mode where the pin actively drives both HIGH and LOW. The default for this driver's outputs.
volatile	C keyword. Prevents the compiler from caching or optimizing away accesses to a variable that hardware can change at any time. All CMSIS register pointers are declared volatile.

1.4. Requirements

The functional and non-functional requirements this module must satisfy are specified in SRS-GPIO_Driver. This document does not restate them; it describes how each one is met at the design and register level.

Traceability: FR-1 through FR-7 map to the seven public functions in Section 4.3. FR-8 (alternate function configuration) maps to gpio_setAlternateFunction(). The private helper is_valid() supports all functions that carry an input-validation requirement.

2. System Architecture

2.1. High-Level Design

- Layered Architecture:
  - Application Layer (main.c): calls gpio_init() once at startup. Delegates LED control to led.c, button reading to button.c, and PWM setup to pwm.c. Has no direct contact with GPIO registers.
  - Middleware (led.c, button.c, pwm.c): translate domain calls (led_toggle, button_get_state, pwm_init) into driver calls. They know the pin assignments (PA5, PC13, PA6) and never access a register directly.
  - GPIO Driver Layer (gpio_driver.c / gpio_driver.h): the module described in this document. Reads and writes GPIO_TypeDef registers directly, validates every argument before touching hardware, and returns a gpio_status_t on every call.
  - Hardware / CMSIS (stm32f4xx.h): supplies GPIO_TypeDef, RCC_TypeDef, and the register constants and bit-mask macros used throughout the driver.

2.2. Module Interfaces

- Public API: the eight functions implemented in gpio_driver.c are declared in gpio_driver.h. Any module that includes the header can call them.
- Dependencies:
  - stm32f4xx.h (CMSIS): provides GPIO_TypeDef, RCC_TypeDef, the peripheral pointers GPIOA-GPIOE and RCC, the clock-enable masks RCC_AHB1ENR_GPIOxEN, and GPIO_TypeDef.AFR[] for alternate function writes.
  - stdint.h: provides uint8_t, uint32_t.
  - stddef.h: provides NULL.

3. Detailed Design

3.1. Class Diagram

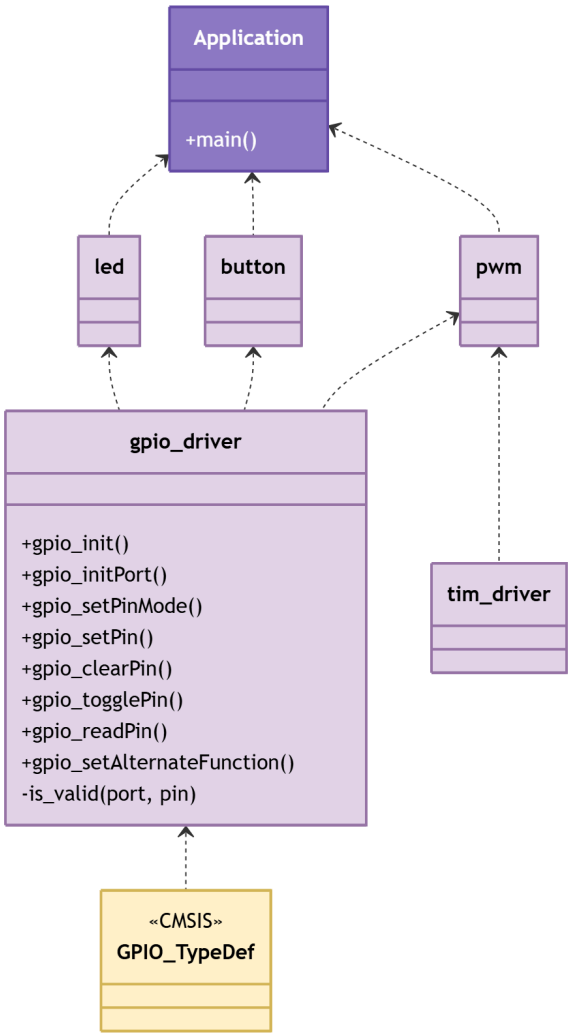

Figure 1. Class Diagram - Driver

3.2. Module Behavior (Sequence Diagrams, State Machines, etc.)

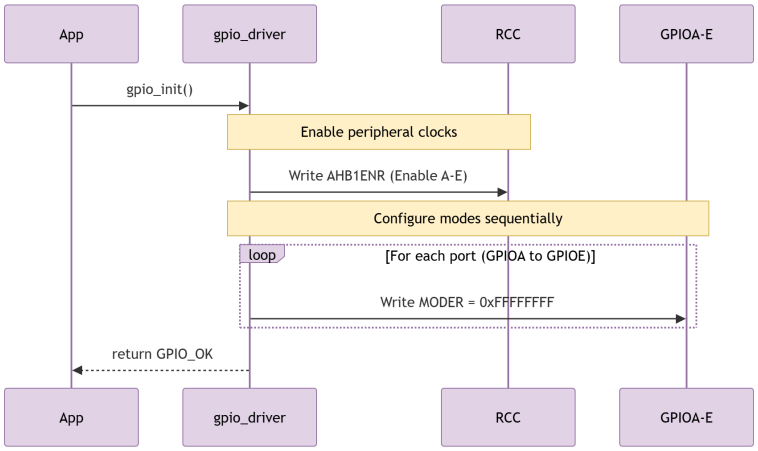

Figure 2. Sequence Diagram - Driver Init

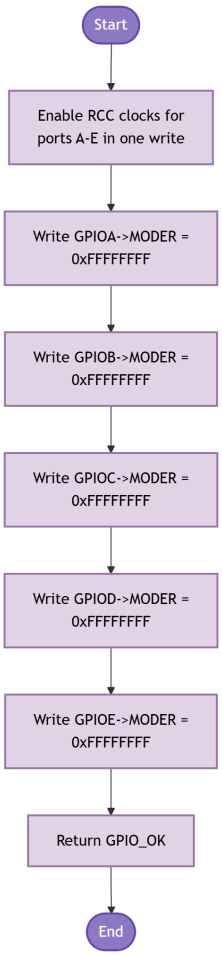

Figure 3. Activity Diagram - Driver Init

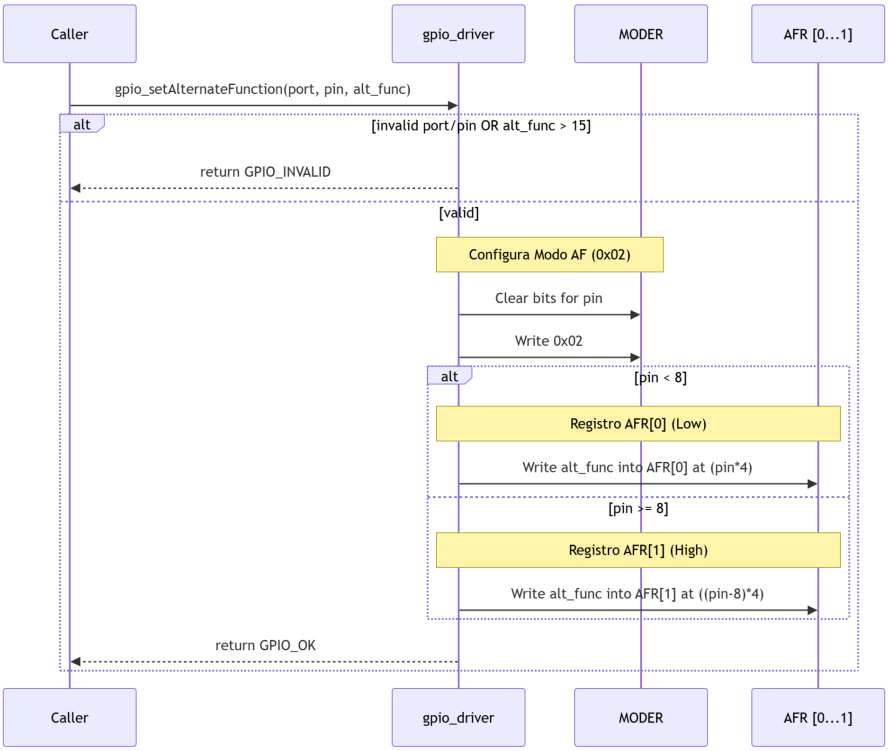

Figure 4. Sequence Diagram - Driver Action with Conditional

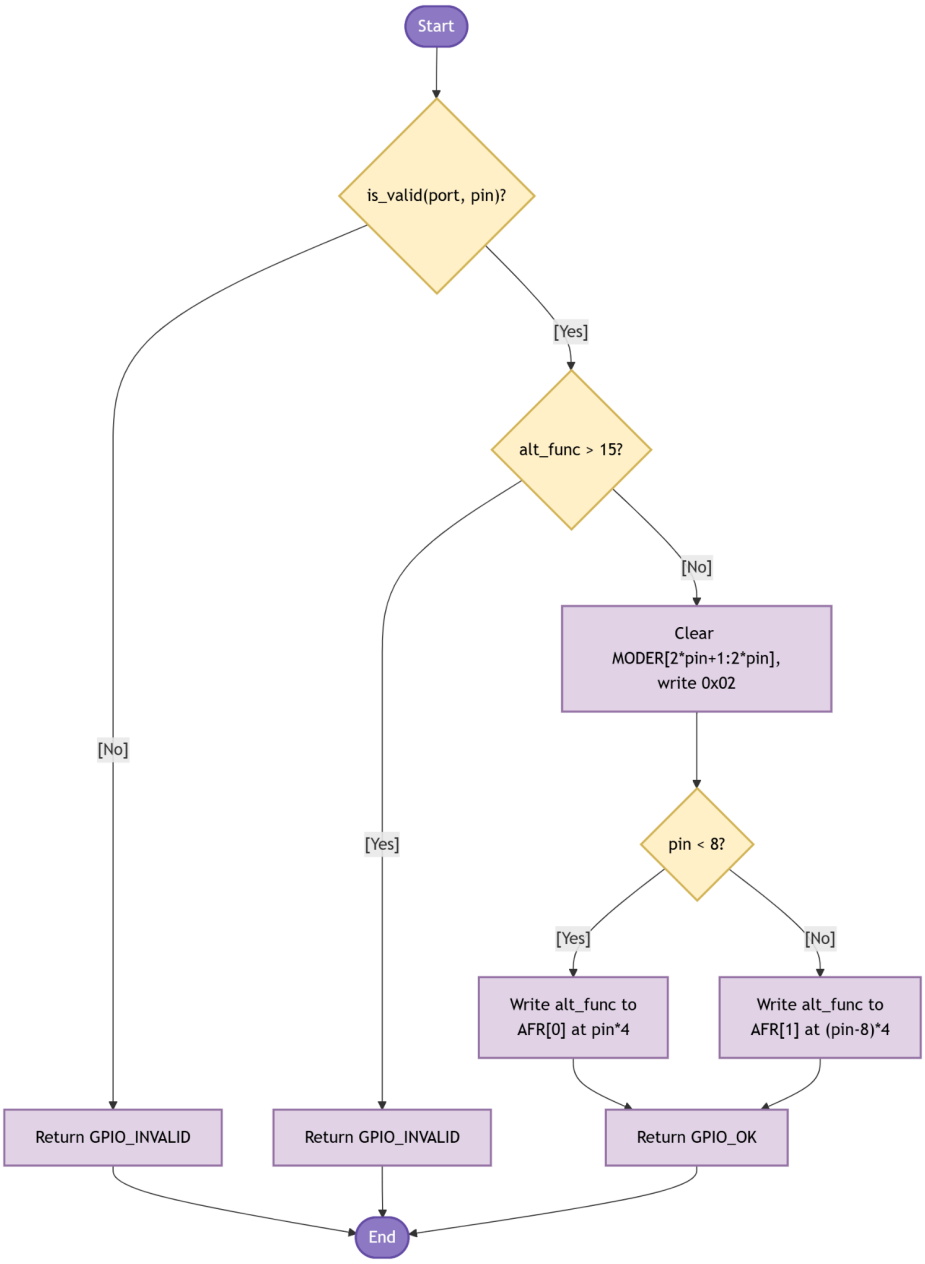

Figure 5. Activity Diagram - Driver Action with Conditional

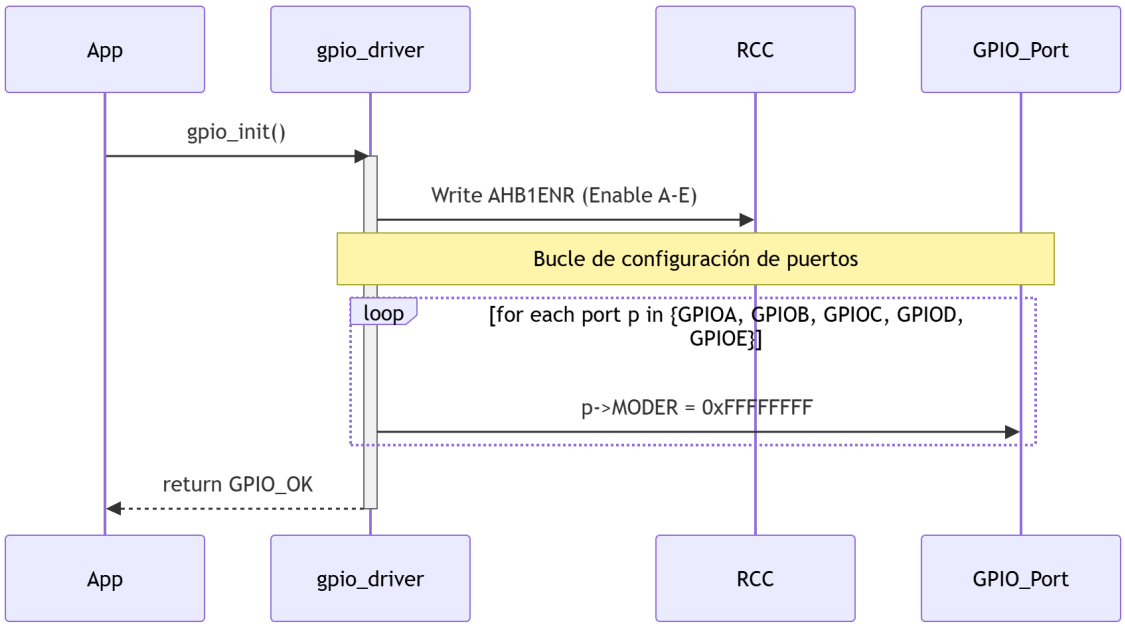

Figure 6. Sequence Diagram - Driver Action with Loop

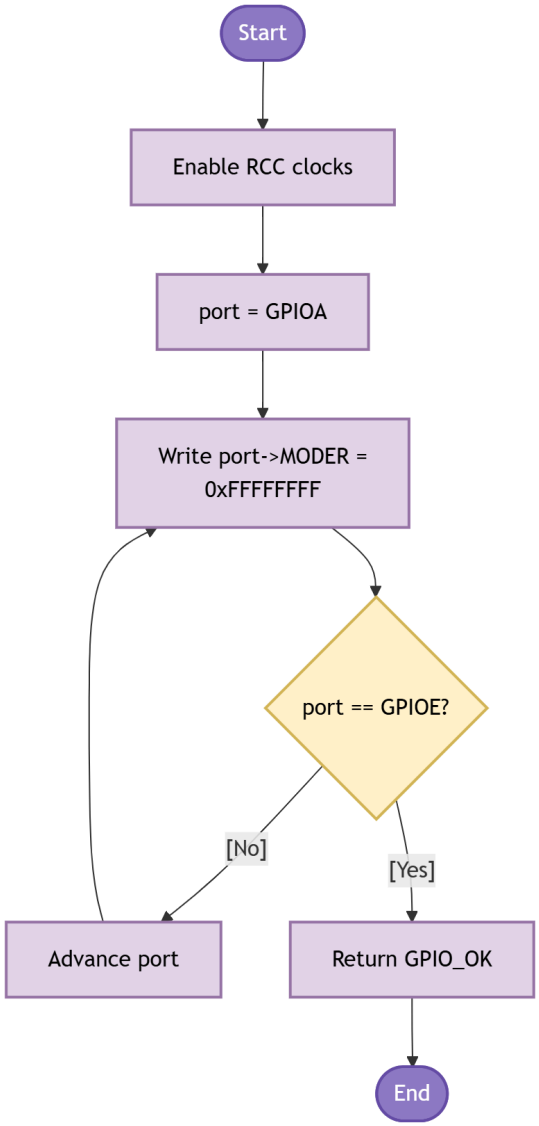

Figure 7. Activity Diagram - Driver Action with Loop

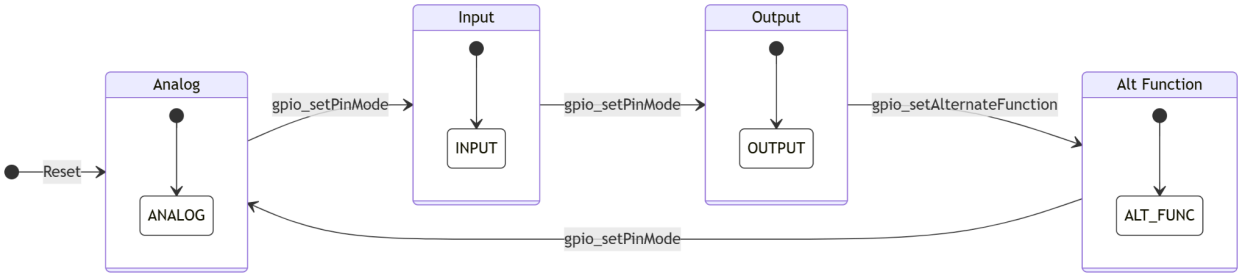

Figure 8. State Diagram - Driver

3.3. GPIO Driver

The driver works in two phases: initialization and per-pin operation. The sections below walk through each function, explain what register it touches and why, and reference the diagrams above.

Initialization

Call gpio_init() once at the start of main(), before any other GPIO function. It does two things in sequence (Figures 2, 3):

- It writes RCC->AHB1ENR in a single OR operation, enabling the peripheral clocks for GPIOA through GPIOE together. A port whose clock is off ignores all register writes, so this must come first.
- It writes 0xFFFFFFFF to the MODER register of each port (Figures 6, 7 show the loop). This places all 16 pins of every port in analog mode, the hardware reset state. Doing it explicitly guarantees a known starting configuration regardless of what the bootloader may have left behind.

If the application only needs one port, call gpio_initPort() instead. It enables the clock for that port alone using an if-else chain that matches the port pointer (GPIOA, GPIOB...) to the correct RCC bit. It does not touch MODER, so any pin configuration already on that port is preserved. Both led.c and pwm.c call gpio_initPort() internally.

Configuring a pin mode

After a port is clocked, call gpio_setPinMode() to set a specific pin. MODER has two bits per pin at positions [2*pin+1 : 2*pin]. The function clears those two bits with an AND-NOT mask, then ORs in the new mode value. Only those two bits change; the other 15 pins are unaffected.

gpio_setPinMode() does not check whether the port clock is on. If the clock is off, the write silently does nothing and GPIO_OK is returned. The caller is responsible for calling gpio_initPort() first.

Driving output pins

gpio_setPin() writes 1 to bit [pin] of BSRR. gpio_clearPin() writes 1 to bit [pin + 16] of BSRR. The hardware drives the pin to the requested state in the same write cycle. Because BSRR is write-only and a single 32-bit write is all that is needed, there is no read-modify-write window. An interrupt that fires between a read and a write on ODR could corrupt the output; BSRR eliminates that risk entirely.

Toggling

gpio_togglePin() reads the current state from ODR, flips the bit with XOR, and writes it back: port->ODR ^= (1U << pin). This is a three-step read-modify-write. In this lab application the main loop is polling-only and no interrupt modifies GPIO ports, so the toggle is safe. If an ISR ever needs to touch the same port, replace the XOR with a conditional set/clear using BSRR.

Reading an input pin

gpio_readPin() samples IDR, which reflects the actual pin voltage at the moment of the read regardless of the pin's configured mode. Before touching IDR the function runs two checks (Figures 4, 5 in the previous version, and is_valid logic):

- is_valid(port, pin): verifies port is not NULL and pin is in [0, 15].
- state != NULL: guards against a hard fault from dereferencing a null pointer.

If either check fails, the function returns GPIO_INVALID without accessing any register. Debounce is the caller's responsibility. IDR captures whatever voltage is on the pin at the instant of the read, so a bouncing contact gives inconsistent results for several milliseconds.

Alternate function configuration

gpio_setAlternateFunction() is new in version 1.1. It does three things in order (Figures 4, 5):

- Validates the arguments: is_valid(port, pin) and alt_func <= 15. Returns GPIO_INVALID on either failure.
- Sets the pin to alternate function mode: clears the 2-bit MODER field and writes 0x02 (0b10).
- Writes the 4-bit AF number into AFR[0] (pins 0-7) or AFR[1] (pins 8-15). For pins 0-7 the field offset is pin * 4. For pins 8-15 the offset is (pin - 8) * 4 within AFR[1]. The function clears the 4-bit slot first, then ORs in the new value.

In this lab pwm.c calls gpio_setAlternateFunction(GPIOA, GPIO_PIN_6, 2) to route TIM3 CH1 to PA6 before the compare channel is enabled. AF2 is the mapping for TIM3 on that pin according to the STM32F411RE datasheet.

Pin mode state machine (Figure 8)

A GPIO pin has four possible modes. After gpio_init() resets MODER to 0xFFFFFFFF, every pin starts in ANALOG mode (bit pair = 0b11). gpio_setPinMode() is the only function that can set INPUT, OUTPUT, or ALT_FUNC modes directly.

gpio_setAlternateFunction() also moves the pin into ALT_FUNC mode as part of its AFR write sequence. No other public function modifies MODER.

In this lab: PA5 (LED) is set to OUTPUT. PC13 (button) is set to INPUT. PA6 (PWM) is set to ALT_FUNC via gpio_setAlternateFunction().

4. Application Programming Interfaces

4.1. Data Types and Macros

4.1.1. Status code

Enumerator	Value
GPIO_OK	Operation completed. All register writes were performed.
GPIO_ERROR	An unspecified hardware or logic occurred.
GPIO_INVALID	One or more arguments failed validation: NULL pointer, unsupported port, pin > 15, invalid mode, or alt_func > 15.

4.1.2. Pin mode enumeration

Enumerator	Value	MODER encoding / Use case
GPIO_MODE_INPUT	0x00	Digital input. Pin floats or follows an external pull resistor.
GPIO_MODE_OUTPUT	0x01	Digital output (push-pull). Pin is driven HIGH or LOW by the driver.
GPIO_MODE_ALT_FUNC	0x02	Alternate function. Pin is controlled by a peripheral (TIM, USART, SPI, etc.). AFR must be configured separately.
GPIO_MODE_ANALOG	0x03	Analog. Default reset state. GPIO input buffer is disconnected; ADC can read the pin.

4.1.3. Pin number enumeration

Enumerator	Value	Meaning
GPIO_PIN_0	0	Pin identifiers for all 16 pins of a GPIO port. Used as the pin argument in all public functions.
GPIO_PIN_15	15	Pin identifiers for all 16 pins of a GPIO port. Used as the pin argument in all public functions.

4.2. Error Codes

All public functions return gpio_status_t. The three defined values are listed in Section 4.1.1. GPIO_INVALID is returned whenever is_valid(port, pin) fails or any additional argument check fails, without modifying any register.

4.3. Function Definitions

Eight functions are exported by gpio_driver.h. Each one calls is_valid() before touching any register, except gpio_init() (which operates on fixed-address constants) and gpio_initPort() (which checks only for a NULL port pointer).

gpio_init

Field	Name / Value	Description
Function Name	`gpio_init`	
Syntax	`gpio_status_t gpio_init(void)`	
Parameters (in)	None	—
Parameters (inout)	None	—
Parameters (out)	None	—
Return value	gpio_status_t	GPIO_OK upon completion. No parameter validation is performed so this function always returns GPIO_OK.

Description	Module initialization. Enables the AHB1 peripheral clocks for GPIOA through GPIOE in a single write to RCC->AHB1ENR. Then writes MODER = 0xFFFFFFFF to each of the five ports, placing all pins in analog mode. Call once at startup before any other GPIO function. In the lab, main.c calls this first.
Constraints	None. Re-calling resets all ports to analog mode.
Available via	gpio_driver.h

gpio_initPort

Field	Name / Value	Description
Function Name	`gpio_initPort`	
Syntax	`gpio_status_t gpio_initPort(GPIO_TypeDef *port)`	
Parameters (in)	port	GPIO_TypeDef* — CMSIS pointer to the target GPIO peripheral (GPIOA … GPIOE).
Parameters (inout)	None	—
Parameters (out)	None	—
Return value	gpio_status_t	GPIO_OK if the clock was enabled. GPIO_INVALID if port is NULL or does not match any of the five supported GPIO base addresses.
Description	Enables the AHB1 peripheral clock for the given port by OR-ing the matching RCC_AHB1ENR_GPIOxEN bit. Does not touch MODER. Used when only one port needs to be clocked, without resetting others. led.c calls this for GPIOA; button.c and pwm.c call it for GPIOC and GPIOA respectively.	
Constraints	Does not configure pin modes. gpio_setPinMode() or gpio_setAlternateFunction() must be called afterward.	
Available via	gpio_driver.h	

gpio_setPinMode

Field	Name / Value	Description
Function Name	`gpio_setPinMode`	
Syntax	`gpio_status_t gpio_setPinMode(GPIO_TypeDef *port, gpio_pin_t pin, gpio_mode_t mode)`	

Parameters (in)	port	GPIO_TypeDef* — pointer to the target GPIO peripheral.
Parameters (in)	pin	gpio_pin_t — target pin number (GPIO_PIN_0 … GPIO_PIN_15).
Parameters (in)	mode	gpio_mode_t — desired mode: GPIO_MODE_INPUT, GPIO_MODE_OUTPUT, GPIO_MODE_ALT_FUNC, or GPIO_MODE_ANALOG.
Parameters (inout)	None	—
Parameters (out)	None	—
Return value	gpio_status_t	GPIO_OK on success. GPIO_INVALID if port is NULL, pin > 15, or mode is out of range.
Description	Clears the two MODER bits for the given pin with an AND-NOT mask (~(0x3U << (pin*2))), then ORs in the new mode value. Only those two bits change; all other pins in the port are unaffected. To configure alternate function mode with a specific AF number, use gpio_setAlternateFunction() instead.	
Constraints	Port clock must already be enabled. If the clock is off, the write is silently ignored.	
Available via	gpio_driver.h	

gpio_setPin

Field	Name / Value	Description
Function Name	`gpio_setPin`	
Syntax	`gpio_status_t gpio_setPin(GPIO_TypeDef *port, gpio_pin_t pin)`	
Parameters (in)	port	GPIO_TypeDef* — pointer to the target GPIO peripheral.
Parameters (in)	pin	gpio_pin_t — target pin number (GPIO_PIN_0 … GPIO_PIN_15).
Parameters (inout)	None	—
Parameters (out)	None	—
Return value	gpio_status_t	GPIO_OK on success. GPIO_INVALID if port is NULL or pin > 15.
Description	Drives the pin HIGH by writing (1U << pin) to BSRR. Bits 0-15 of BSRR set the corresponding output bit. The write is atomic; no read-modify-write is needed, so an interrupt cannot cause a race on the port's output register.	

Constraints	Pin must be configured as OUTPUT or ALT_FUNC. Calling on an INPUT pin is allowed but has no visible effect on the external signal.
Available via	gpio_driver.h

gpio_clearPin

Field	Name / Value	Description
Function Name	`gpio_clearPin`	
Syntax	`gpio_status_t gpio_clearPin(GPIO_TypeDef *port, gpio_pin_t pin)`	
Parameters (in)	port	GPIO_TypeDef* — pointer to the target GPIO peripheral.
Parameters (in)	pin	gpio_pin_t — target pin number (GPIO_PIN_0 … GPIO_PIN_15).
Parameters (inout)	None	—
Parameters (out)	None	—
Return value	gpio_status_t	GPIO_OK on success. GPIO_INVALID if port is NULL or pin > 15.
Description	Drives the pin LOW by writing (1U << (pin + 16)) to BSRR. Bits 16-31 of BSRR reset the corresponding output bit. The write is atomic. gpio_init() calls this to ensure the LED starts off.	
Constraints	Same constraints as gpio_setPin: pin must be in output mode and port clock must be enabled.	
Available via	gpio_driver.h	

gpio_togglePin

Field	Name / Value	Description
Function Name	`gpio_togglePin`	
Syntax	`gpio_status_t gpio_togglePin(GPIO_TypeDef *port, gpio_pin_t pin)`	
Parameters (in)	port	GPIO_TypeDef* — pointer to the target GPIO peripheral.
Parameters (in)	pin	gpio_pin_t — target pin number (GPIO_PIN_0 … GPIO_PIN_15).
Parameters (inout)	None	—

Parameters (out)	None	—
Return value	gpio_status_t	GPIO_OK on success. GPIO_INVALID if port is NULL or pin > 15.
Description	Flips the output bit by XOR-ing ODR: port->ODR ^= (1U << pin). This is a read-modify-write sequence and is not atomic. Safe in this lab because the main loop is polling-only and no ISR writes to the same port. If an interrupt ever needs to modify the same port output, replace this with a BSRR-based conditional set/clear.	
Constraints	Not atomic. Unsafe in preemptive or interrupt-driven contexts where another execution context modifies the same port.	
Available via	gpio_driver.h	

gpio_readPin

Field	Name / Value	Description
Function Name	`gpio_readPin`	
Syntax	`gpio_status_t gpio_readPin(GPIO_TypeDef *port, gpio_pin_t pin, gpio_pin_state_t *state)`	
Parameters (in)	port	GPIO_TypeDef* — pointer to the target GPIO peripheral.
Parameters (in)	pin	gpio_pin_t — target pin number (GPIO_PIN_0 … GPIO_PIN_15).
Parameters (inout)	None	—
Parameters (out)	state	gpio_pin_state_t* — pointer to a variable that will receive GPIO_PIN_LOW (0) or GPIO_PIN_HIGH (1).
Return value	gpio_status_t	GPIO_OK on success. GPIO_INVALID if port is NULL, pin > 15, or state is NULL.
Description	Reads bit [pin] from IDR and writes the result into *state. IDR always reflects the actual pin voltage at the instant of the read, regardless of the pin's configured mode. The state == NULL check guards against a hard fault. Debounce is the caller's responsibility.	
Constraints	No debounce is performed. For the user button (PC13, active LOW), button.c inverts the polarity: GPIO_PIN_LOW maps to BUTTON_PRESSED.	
Available via	gpio_driver.h	

gpio_setAlternateFunction

Field	Name / Value	Description

Function Name	`gpio_setAlternateFunction`	
Syntax	`gpio_status_t gpio_setAlternateFunction(GPIO_TypeDef *port, gpio_pin_t pin, uint8_t alt_func)`	
Parameters (in)	port	GPIO_TypeDef* — pointer to the target GPIO peripheral.
Parameters (in)	pin	gpio_pin_t — target pin number (GPIO_PIN_0 … GPIO_PIN_15).
Parameters (in)	alt_func	uint8_t — alternate function number (0–15) as defined in the STM32F411RE datasheet for the target pin.
Parameters (inout)	None	—
Parameters (out)	None	—
Return value	gpio_status_t	GPIO_OK on success. GPIO_INVALID if port is NULL, pin > 15, or alt_func > 15.
Description	Configures a pin for alternate function output in three steps: (1) validates all arguments; (2) clears the 2-bit MODER field for the pin and writes 0x02 (alternate function mode); (3) writes the 4-bit AF number into AFR[0] for pins 0-7, or AFR[1] for pins 8-15, clearing the slot first. In this lab, pwm.c calls gpio_setAlternateFunction(GPIOA, GPIO_PIN_6, 2) to route TIM3 CH1 to PA6 before enabling the compare channel.	
Constraints	The port clock must be enabled before calling. The AF number must match the datasheet mapping for the target peripheral on that pin.	
Available via	gpio_driver.h	

4.4. Internal Function Definitions

is_valid

Field	Name / Value	Description
Function Name	`is_valid`	
Syntax	`static int is_valid(GPIO_TypeDef *port, gpio_pin_t pin)`	
Parameters (in)	port	GPIO_TypeDef* — port pointer to validate.
Parameters (in)	pin	gpio_pin_t — pin number to validate.
Parameters (inout)	None	—

Parameters (out)	None	—
Return value	int	1 if port is not NULL and pin <= 15. 0 otherwise.
Description	Validates that the port pointer is not NULL and the pin number is within the valid range [0, 15]. Returns 0 on any failure; the calling public function then returns GPIO_INVALID without touching any register. The implementation is two lines: a NULL check and a pin > 15 check.	
Constraints	Does not verify that port points to a supported port address (GPIOA-GPIOE). A caller could pass an arbitrary non-NULL address and pass this check.	

5. Future Enhancements

These features are outside the current scope but are worth adding in future iterations:

- **Interrupt support:** configure EXTI lines via SYSCFG and NVIC to trigger callbacks on rising, falling, or both edges. Requires two new functions: gpio_enableInterrupt() and a user-supplied callback registration.
- **Open-drain output mode:** the OTYPER register (offset 0x04) controls push-pull vs. open-drain per pin. A gpio_setOutputType() function would expose this, needed for I2C lines and wired-OR bus topologies.
- **Pull-up / pull-down control:** the PUPDR register (offset 0x0C) enables internal pull resistors (00 = none, 01 = pull-up, 10 = pull-down). A gpio_setPullMode() function would avoid relying on external resistors for floating inputs.
- **Output speed control:** the OSPEEDR register (offset 0x08) sets slew rate (low / medium / fast / very high). Higher frequencies (SPI, SDIO) need the higher settings to keep signal edges clean.
- **Port-level read/write:** gpio_writePort(port, mask, value) and gpio_readPort(port, *value) functions would allow setting or sampling multiple pins in a single register access, useful for parallel bus interfaces.